

数字图像处理—边界跟踪算法Suzuki85



德怀特

巅峰单换勒布朗

📖 收录于 · 数字图像处理 >

43 人赞同了该文章 >

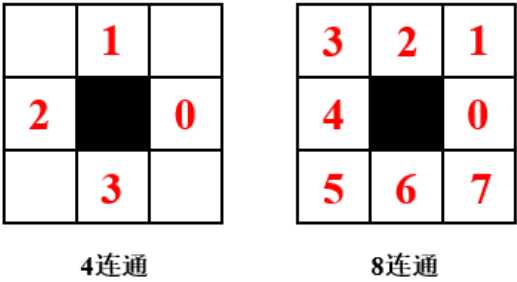
本文是Suzuki85+的原理解析

原论文：Suzuki S. Topological structural analysis of digitized binary images by border following[J]. Computer vision, graphics, and image processing, 1985, 30(1): 32-46.

算法功能：将一个二值图像转化成边界的表示形式，并且提取各边界之间的拓扑结构

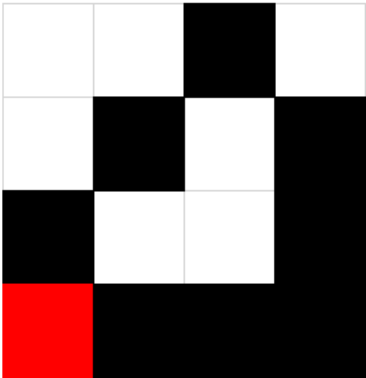
概念：链码

链码主要用来描述边界或者曲线，利用起始点坐标和边界点方向代码。常用的链码分为4连通链码和8连通链码，4连通代表只有上下左右四个方向的像素与中心像素相连，属于一个连通域，即属于一条边界或曲线；8连通代表8个方向与中心像素相连。



通常使用8连通，此时“偶数”码（0,2,4,6）单位长度为1，奇数码（1,3,5,7）长度为根号2

假设某物体的轮廓如下面的二值图像所示，红色代表链码的起始点，设坐标为（1,1）



1，freeman链码（原链码）

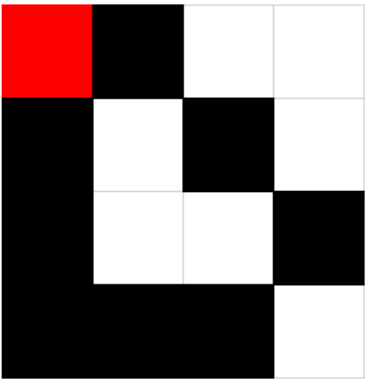
从起始点开始，记录起始点坐标，以8连通的方式从编码为1的方向（45度）开始搜索下一个边界像素点。找到之后记录方向编码（0到7），然后从找到的像素开始重复以上步骤，得出一组链码，即为freeman链码。如果是闭合的，编码完成后返回到起始点，则可以省略起始点坐标。上图的freeman码是（1,1）21176644或者211766444

2，归一化链码

原链码具有平移不变性（平移时不改变指向符），但当改变起点S时，会得到不同的链码表示，即不具备唯一性。将链码看作n位自然数，将该码按一个方向循环，使其构成的n位自然数最小，此时就形成起点唯一的链码，称为归一化链码。上图的归一化链码为117664442

3，一阶差分链码

归一化链码解决了因为起点坐标不同而编码不同的问题，但是当边界发生旋转，归一化链码仍然会发生变化



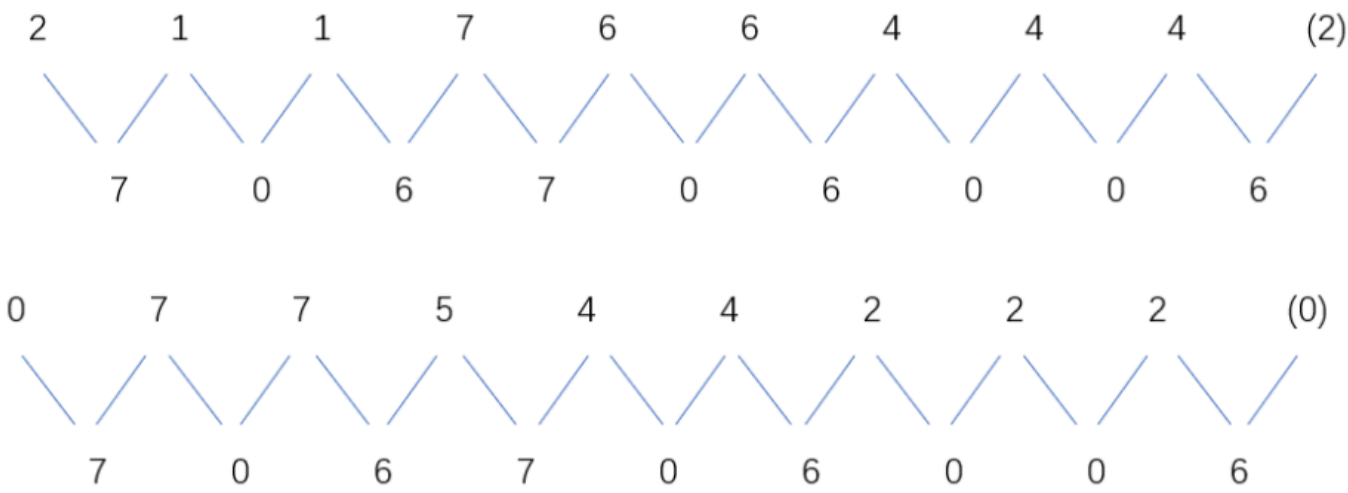
链码为666001334（按未旋转前链码编码的顺序），归一化链码为001334666，旋转后链码编码发生了改变，归一化链码也发生改变

这时候可以用一阶差分链码，原理是计算相邻两个元素方向变化（按逆时针方向）的次数

3	2	1
4		0
5	6	7

例如7到6，变化了7次，记为7；6到7辩护了一次，记为1

上图旋转前后的一阶差分链码都为706706006



其实方向编码2和0要用两次

4，归一化一阶差分链码

即对一阶差分链码归一化，得出的链码具有平移不变性和旋转不变性。上图的归一化一阶差分链码为006706706。

算法：Suzuki85

1，算法简介

Suzuki85是边界跟踪算法（border following algorithm）的拓展版本，用于数字化二值图像的拓扑结构分析（the topological analysis of digitized binary images），主要获取二值图像的边界（outer border和hole border）的包围关系（surroundness relations）

相对于边界跟踪算法的拓展：（1）给每个边界不同的标记；（2）增加一个步骤，用来获得当前被跟踪的边界的父/母边界（parent border）

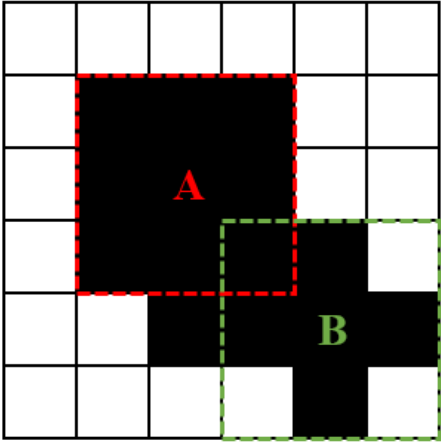
Suzuki85的修改版：仅跟踪最外层边界（outermost border）（即该边界之外无包围它的孔边界）

Suzuki85的优势：既可以完成**边界跟踪**，又可以实现图像的**拓扑结构分析**，利用Suzuki85可以将原始二值图像用边界和包围关系代替，简化二值图像存储方式，提升二值图像处理效率

2，相关定义

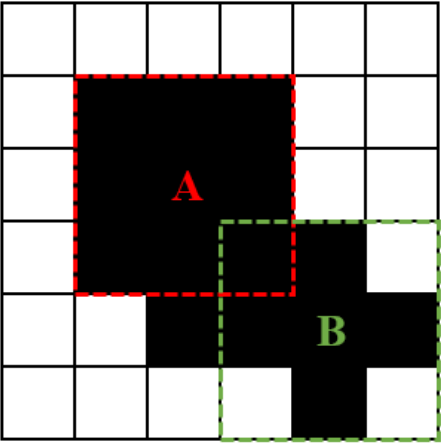
- 边界点 (border point) 的定义

在4-连通的情况下（1-像素点4-连通），一个1-像素点(i,j)在它的8-邻域内存在一个0-像素点，则称该1-像素点为边界点。如下图，所有黑点满足4-连通，视为一个连通域。A点的8-邻域内不存在0-像素点，所以A点不是边界点；B点的8-邻域内存在0-像素点，所以**B点是边界点**。所以某类像素点4-连通，则该类像素点必须要被8个同类像素点包围才不会被看作边界点（**4-连通要求严格**）



黑点该表1-像素；白点代表0-像素；1-像素4-连通，则A不是边界点，B是边界点

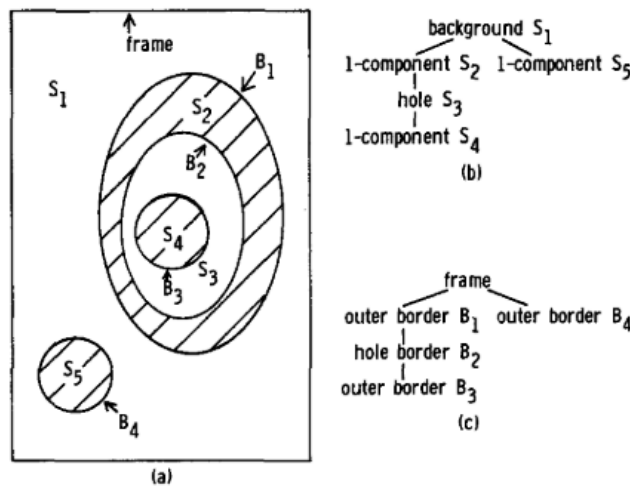
在8-连通的情况下（1-像素点8-连通），一个1-像素点(i,j)在它的4-邻域内存在一个0-像素点，则称该1-像素点为边界点。如下图，所有黑点满足8-连通，视为一个连通域。A点的4-邻域内不存在0-像素点，所以A点不是边界点；B点的4-邻域内不存在0-像素点，所以**B点也不是边界点**（8-连通要求不严格）



黑点该表1-像素；白点代表0-像素；1-像素8-连通，则A不是边界点，B也不是边界点

综上，一个点是不是边界点，需要先明确连通的类型

- 边框、包围关系、外边界和孔边界定义



连通域之间的包围关系和边界之间的包围关系

(1) 边框 (frame)：最上行、最下行、最左行和最右行组成，定义边框的像素为0 (0-pixels)

(2) 包围关系(surroundness relation): 如上图，S1直接包围S2，S2直接包围S3，S3直接包围S4，S1直接包围S5

(3) 外边界(outer border): 一个任意的1-连通域和直接包围它的0-连通域之间的边界点组成的集合

(4) 孔边界(hole border): 一个孔和直接包围它的1-连通域之间的边界点的集合

注意，**边界（包括孔边界和外边界）是用1-像素定义的**，不同于边框和背景的0-像素

• 父/母边界 (parent border) 定义

(1) outer border的parent border: 如上图，B3是一个outer border，它的母边界就是B2；B1是一个outer border，它的母边界就是frame

(2) hole border的parent border: 如上图，B2是一个hole border，它的母边界是B1

parent border的定义实际上就是外层的是里一层的parent

• 边界包围 (surroundness among borders) 定义

二值图像中，两个给定的边界 B_0 和 B_n ，我们说 B_n 包围 B_0 是指，存在一系列边界 $B_0, B_1 \dots B_n$ ，对所有的 k ， B_k 是 B_{k-1} 的母边界

• 坐标体系——横坐标从左往右递加，纵坐标从上往下递加

• 连通约定——当0-pixels使用8-连通，1-pixel必须使用4-连通；当0-pixels使用4-连通，1-pixels必须使用8-连通，以避免topological contradiction

3, 相关属性

• 属性 (**边界的唯一性**)：二值图上，对任意一个1-连通域，它的外边界是1且是唯一的，对任意孔，它的边界(该hole和直接包围它的1-component之间的边界)也是唯一的

4, 算法流程

第一步：首先对输入的二值图像执行光栅扫描 (raster scan)，从上到下，从左到右，寻找满足边界条件的点，即寻找**边界跟踪起始点** (the border following starting point)，**如果该点同时满足外边界起始点和孔边界起始点的条件，则将该点看作外边界跟踪起始点!!!**为每一个新找到的边界赋不同的数字标号

NBD: 最新找到的边界 B 的数字标号 (当前边界)

LNBD: 上一个找到的边界 B' 。它要么是当前边界的母边界，要么和当前边界拥有同一个母边界

如下表，新边界是 B ，上一个边界是 B' ，根据 B 和 B' 的边界类型确定 B 的parent border

Decision Rule for the Parent Border of the Newly Found Border B

Type of the border B' with the sequential number LNBD		
Type of B	Outer border	Hole border
Outer border	The parent border of the border B'	The border B'
Hole border	The border B'	The parent border of the border B'

确定新边界的parent border的规则

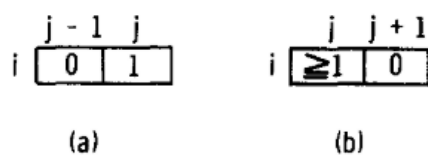
第二步：找到边界跟踪起始点后，执行边界跟踪，找到完整边界。边界跟踪的策略和经典的类似，但在标记策略上有所不同

(a) 外边界最右边的像素点被标记成-NBD

(b) 其余情况下，除非该边界点已经被跟踪过（已经属于其它边界），否则都标记为NBD（也就是说如果该边界点已经被跟踪过，已经属于其它边界，但是又属于该边界，就赋值为-NBD???)

一旦出现-NBD，该点的边界标号永远不得修改，不得被判断为起始点，因此 (a) 和 (b) 可以避免这些标记点被再次认作hole border和outer border的起始点

当跟踪和标记完了整条边界，重新开始光栅扫描，当扫描完图片右下角最后一个点，算法结束



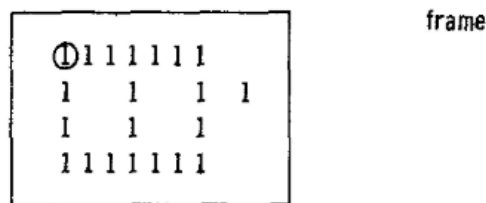
(1)外边界起始点条件; (b)孔边界起始点条件

实例分析

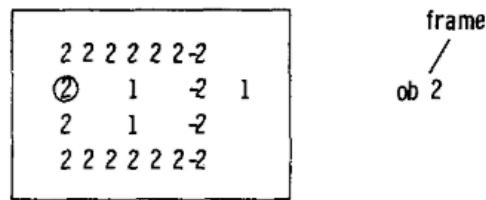
如下图，(a) 是输入图像，当光栅扫描到圈住的1-pixel时，发现其满足外边界起始点的条件，此时停止扫描，进行边界跟踪。此边界的数字标号为2和-2（1留给图片的框架），同时确定该边界的parent border是frame。当边界跟踪完成，继续光栅扫描，遇到新的起始点再停止扫描.....直到扫描到右下角最后一个点。

每次扫描新的一行时，都将LNDB置为1，即frame要永远视为根边界

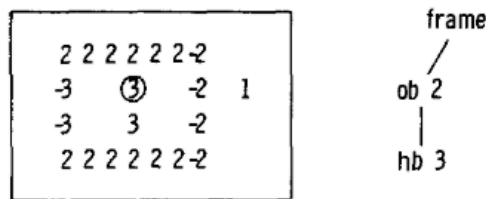
下图是一个具体实例，因为外边界和孔边界存在边界点共用的情况，所以看起来有一些混乱，究竟什么时候要赋NBD？什么时候赋-NBD？什么时候不允许修改？



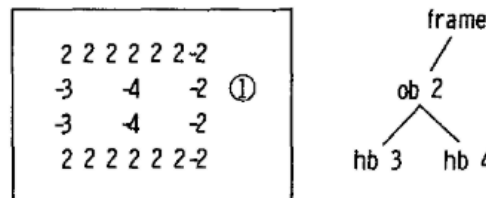
(a)



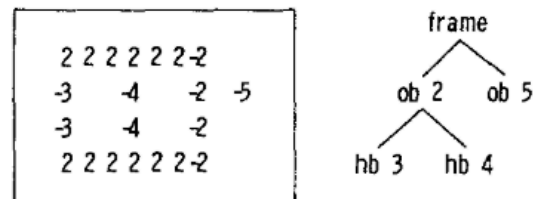
(b)



(c)



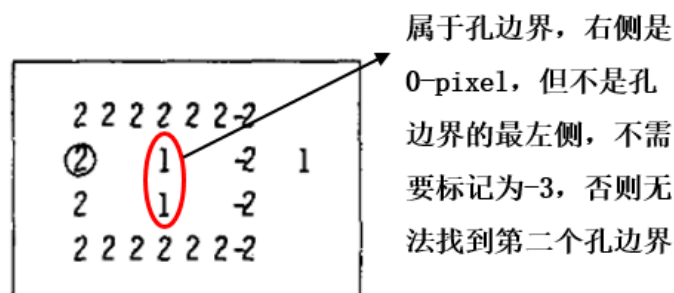
(d)



右侧是边界之间的逻辑结构，上层是下层的parent border

想要解决上述困惑需要搞清楚两个问题

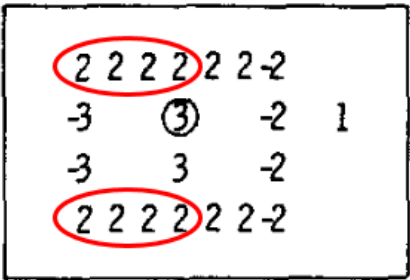
(1) **为什么要使用-NBD?** 答案是为了防止后续扫描将这些点误判为起始点。那么哪些点可能会存在误判的情况? —— (1) **外边界的最右侧的点** (即属于外边界, 右边又是0-pixel的点)。这些点如果不标记为-NBD, 后续会被误判为孔边界的起始点, 而实际上这些点即使也属于某孔边界, 也不可能是其起始点, 一个边界只需要一个起始点; (2) **孔边界的最左侧的点** (即属于孔边界, 右边又是0-pixel的点)。这些点同样可能被误判为孔边界的起始点。但是这种情况却有一个例外, 如下图



因此遇到上述两种类型的边界点, 需要将其赋值为-NBD。这一切的**祸源**都来自于孔边界起始点的条件 (某点标号**大于等于1**且右边是0-pixel), 那如果把孔边界起始点的条件修改为标号等于1且右边是0-pixel不就可以避免-NBD的使用了? 但是这样就无法找到某些孔边界了, 比如上述实例中的孔边界。

(2) 什么情况下不允许修改标号?

并不是所有孔边界的点都被标记，如下图，有八个点的边界标号没有被修改。这里制定了一个规则——边界点右侧不是0-pixel，且原标号不等于1，则不修改。因此下图中圈住的八个点都不需要修改边界标号



边界跟踪算法流程

从边界起始点开始，如何找到完整的一条边界？

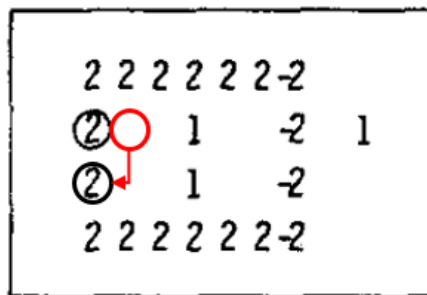
该点是边界的最后一个点

从起始点左侧的点开始，先顺时针在起始点的邻域(8/4邻域)中找第一个非零点(即边界的最后一个点)

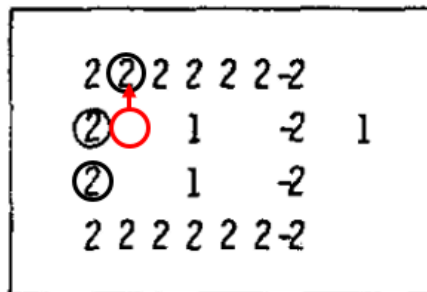
从起始点左侧的点开始，再逆时针在起始点的邻域中找第一个非零点(即边界的第二个点)

以上一步找到的边界的第二个点为中心，第一个点左侧开始逆时针找中心点领域中的第一个非零点(即边界的第三个点)

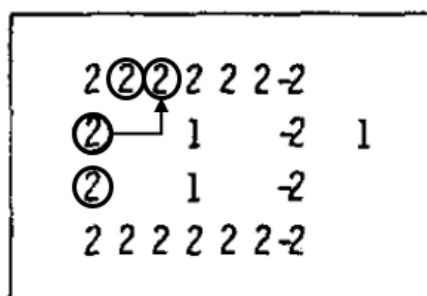
outer border的边界跟踪流程的前三步，后面以此类推，指导重新回到起始点，跟踪完毕。黑圈代表已经找到的边界点



从起始点右侧的点开始，先**顺时针**在起始点的**邻域**中找第一个非零点(即边界的最后一个点)



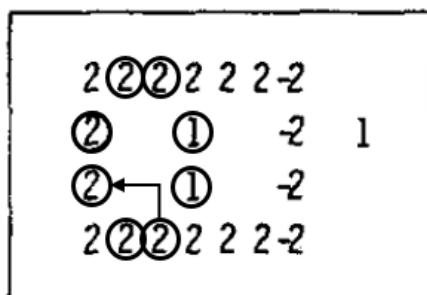
从起始点右侧的点开始，再**逆时针**在起始点的**邻域**(这里默认是8邻域)中找第一个非零点(即边界的第二个点)



以上一步找到的边界的第二个点为中心，第一个点左侧开始**逆时针**找中心点领域中的第一个非零点(即边界的第三个点)

hole border的边界跟踪流程的前三步，后面以此类推，指导重新回到起始点，跟踪完毕。黑圈代表已经找到的边界点

需要特别注意，不管是outer border还是hole border，只有在第一步中才顺时针搜索，之后不断更换中心点，始终使用逆时针搜索非零点



此时搜索到的非零点是最后一个边界点，搜索结束（该情况默认是8邻域）

参考：

[freeman链码，归一化链码，一阶差分链码，归一化一阶差分链码_ok460695972ko的博客-CSDN博客_归一化链码](#)

[freeman链码_Apikaqiu的博客-CSDN博客_freeman链码](#)

[Opencv轮廓跟踪算法源码分析（1）--icvFetchContour\(\)_yiqiudream的博客-CSDN博客](#)

发布于 2021-08-09 16:25

数字图像处理（书籍）

来源：知乎 | 悠趣谷·零成本 | 我要插件